Adaptive Boss Agent - Utility AI Design

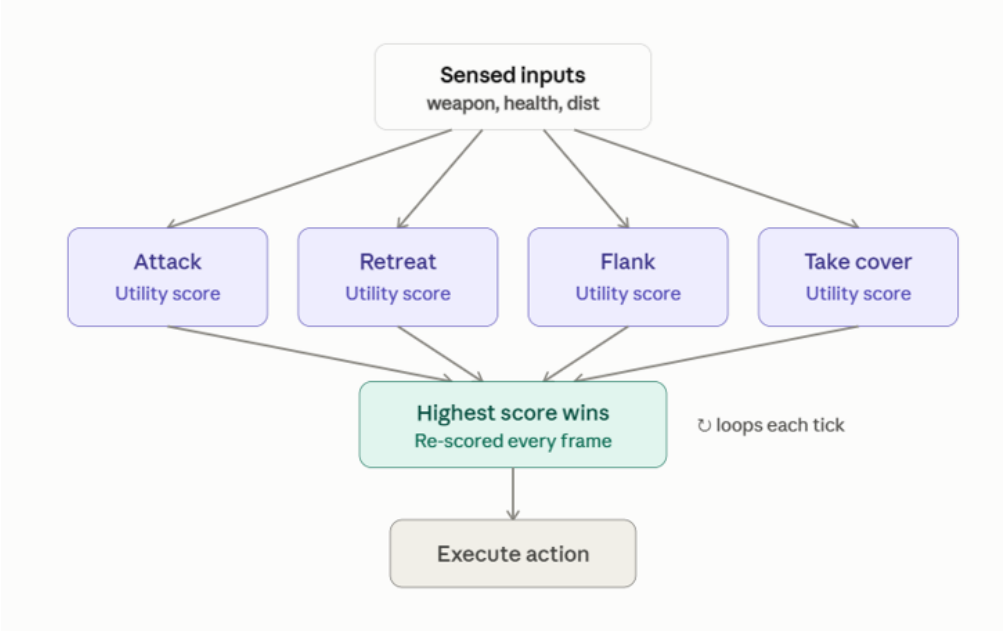
SE3062 Intelligent Systems | Agent owner: Suwasthikka S (IT24101502)

1. Overview

The Adaptive Boss uses a Utility AI architecture. Every decision cycle, it scores four tactical actions from the current game state and performs the highest-scoring one. Unlike a Finite State Machine, there are no fixed transitions: the boss re-evaluates all options continuously, so its tactics change as the player's weapon, the player's health, its own health, and the distance change.

The decision logic is a plain C# class (BossBrain) that knows nothing about GameObjects. A separate MonoBehaviour (BossController) supplies inputs and carries out the chosen action, which keeps AI logic decoupled from visuals.

2. Architecture Diagram



3. Inputs

Input	Range	Notes
Player weapon type	Melee / Ranged	Converted to a ranged-threat value: 0.9 if ranged, 0.1 if melee
Player health	0.0 - 1.0	Fraction of maximum health
Boss health	0.0 - 1.0	Fraction of maximum health
Distance to player	meters	Straight-line distance; 20 m or more counts as far

4. Considerations

A consideration reads one input, applies a response curve, and outputs a score between 0 and 1. Each action below is a weighted average of its considerations.

Consideration	Formula	Curve/meaning
Closeness	1 - clamp(d / 20)	Linear: 1 when touching, 0 at 20 m or more
Medium band	1 - clamp(|d - 10| / 10)	Triangle peaking at 10 m: the ideal flanking range
Boss hurt	1 - bossHealth	Linear
Retreat urgency	(1 - bossHealth)^2	Quadratic: stays low until the boss is badly hurt, then rises fast
Player hurt	1 - playerHealth	Linear: a weak player invites aggression
Ranged threat	0.9 ranged / 0.1 melee	Step: the weapon type changes the whole tactic

5. Actions and Weights

Action	Score (weights sum to 1)	Why these weights
Attack	0.4 x bossHealth + 0.4 x closeness + 0.2 x playerHurt	Health and closeness matter equally, since an attack needs both strength and reach. Player weakness is a smaller bonus.
Retreat	0.7 x retreatUrgency + 0.3 x closeness	Low health is the main reason to flee. Closeness adds urgency, so a hurt boss far from the player is not forced to run.
Flank	0.5 x rangedThreat + 0.5 x mediumBand	Flanking is worth it against ranged weapons, and only at a range where circling is possible.
Take Cover	0.6 x rangedThreat + 0.4 x bossHurt	Cover is mainly a response to ranged weapons. Being hurt makes hiding more attractive.

The weights are initial estimates chosen from the reasoning above. They are tuned by play-testing, and the debug overlay shows all four live scores so each change can be justified.

6. Decision Rule (anti-flicker)

Because scores are recomputed every frame, two close scores could make the boss switch action many times per second. To prevent this, the boss commits to an action:

- It holds the current action for at least 1.0 second (minCommitTime).
- After that, it switches only if the best action beats the current one by more than 0.15 (switchMargin).

7. Worked Example

Situation: boss health 30%, player health 80%, player has a ranged weapon, distance 14 m.

Considerations: closeness = 1 - 14/20 = 0.30; medium band = 1 - |14 - 10|/10 = 0.60; boss hurt = 0.70; retreat urgency = 0.70^2 = 0.49; player hurt = 0.20; ranged threat = 0.9.

Action	Calculation	Score
Attack	0.4(0.30) + 0.4(0.30) + 0.2(0.20)	0.28
Retreat	0.7(0.49) + 0.3(0.30)	0.43
Flank	0.5(0.90) + 0.5(0.60)	0.75
Take Cover	0.6(0.90) + 0.4(0.70)	0.82

Result: Take Cover scores highest. The player has a ranged weapon, and the boss is badly hurt, so attacking directly would be risky.

Anti-flicker in action: if the boss was already flanking (0.75), Take Cover leads by only 0.07, which is under the 0.15 margin, so the boss keeps flanking. It would switch only if Take Cover rose above 0.90.

8. Why Utility AI

Alternative	Why Utility AI was preferred
Finite State Machine	Needs a hand-written transition for every pair of states (four actions means up to twelve). Utility AI has none: adding an action means adding one scoring function.
Behaviour Tree	Its priority order is fixed, so it cannot easily weigh several factors at once, such as weapon type together with health. Utility scores blend them smoothly.
GOAP	Plans multi-step sequences, which is more than this boss needs. It has a higher implementation cost and is harder to debug for four single-step tactics.

Utility AI directly supports the design goal of changing tactics based on the player's weapon and health, and the numeric scores make each decision explainable and inspectable.

Trade-offs: weights need tuning by play-testing, and scores can be hard to predict when many considerations interact. This is mitigated by keeping considerations few and simple, by the on-screen debug scores, and by the anti-flicker rule.

9. Mapping to Code

File	Responsibility
BossBrain.cs	Pure logic: considerations, action scores, anti-flicker rule. No Unity objects.
BossController.cs	Gathers inputs, calls BossBrain.Evaluate(), performs the movement, shows the debug overlay.